DGAD: Disagreement-Gated Adaptive Detection

How It Differs from Prior Work -- Complete Comparative Analysis

Project: Adversarial Attack Detection for LLMs (SVKM NMIMS Capstone, Sem 7)

Team: Rajpreet Singh Khurana (K033), Dhruv Rathod (K055), Sumit Pandey (K044)

Mentor: Dr. Ruchi Sharma

Date: September 2026

---

1. Executive Summary

DGAD (Disagreement-Gated Adaptive Detection) is a **black-box, model-agnostic security layer** that sits in front of any LLM and inspects every prompt before it reaches the model. Its core novelty: **disagreement between cheap detectors is used as a routing signal**, not averaged into a single score.

When cheap channels (statistical, semantic, offset) **agree** -> immediate PASS or BLOCK.

When they **disagree** -> escalate *only that prompt* to expensive verifiers (behavioural probe + LLM-as-judge).

A **self-adversarial calibration loop** continuously hunts for inputs that fool all cheap channels in the same direction (false agreement) and converts them into training data.

This document compares DGAD against 20 relevant papers from the literature review.

---

2. Prior Work Landscape -- Categorized

Category	Papers	Core Approach
Optimization Attacks	Zou et al. (GCG, 2023), Liu et al. (AutoDAN, 2023), Su (CoT-GCG, 2024)	Generate adversarial suffixes / fluent jailbreaks
Prompt Injection	Perez & Ribeiro (2022), Liu et al. (Formal Framework, 2023), Yi et al. (Indirect Injection Benchmark, 2023)	Taxonomy, benchmarks, threat models
Statistical Detection	Alon & Kamfonas (Perplexity, 2023), Jain et al. (Baseline Harness, 2023)	Windowed perplexity under GPT-2
Semantic/Embedding Detection	Li & Liu (InjecGuard, 2024), Sekar et al. (ZEDD, 2026)	Embedding classifiers, drift detection
White-Box / Gradient	Xie et al. (GradSafe, 2024)	Safety-critical gradient analysis
LLM-as-Judge / Guardrails	Inan et al. (Llama Guard, 2023), Zizzo et al. (Guardrail Benchmark, 2025)	Instruction-tuned safety classifiers
Ensemble / Voting	Appleroll (PromptForest, 2026), Robey et al. (SmoothLLM, 2023/25)	Weighted voting, randomized smoothing
Cascaded / Multi-Layer	Abasikeles-Turgut & Gumus (CASCADE, 2026)	Regex -> Embedding -> LLM fallback
Mutation-Based	Zhang et al. (JailGuard, 2023/25)	Input perturbation -> KL divergence on outputs
Near-Zero-Cost	Wang et al. (FJD, 2025)	Affirmative prefix + temperature-scaled logits
Disagreement / Uncertainty	Jiang et al. (DiscoUQ, 2026)	Disagreement structure -> selective prediction
Adaptive Evasion	Hackett et al. (Bypassing Guardrails, 2025)	Character injection + AML obfuscation against detectors

---

3. DGAD Architecture at a Glance

```
```


INPUT PROMPT
NORMALISATION (unicode, de-obfuscation)
Channel A
Statistical
(Perplexity)
Channel B
Semantic
(Embedding)
Channel E
Offset
(Rep.Offset)
CALIBRATION LAYER (Platt / Isotonic)
Raw scores -> Calibrated probabilities [0,1]
DISAGREEMENT GATE (Core Novelty)
Agreement (all PASS or all BLOCK) -> DECIDE
Disagreement -> ESCALATE to Channels C + D
Channel C
Behavioural
(Perturb-
consistency)
Channel D
LLM Judge
(Llama
Guard /
Custom)
FINAL DECISION
+ RATIONALE

AUDIT CANARY
LOG (Indirect
(PostgreSQL) Injection)

SELF-ADVERSARIAL
CALIBRATION LOOP
(Continuous red-
teaming own
cheap channels)

```

---

## 4. Detailed Comparison: DGAD vs. Each Prior Paper

### *4.1 vs. DiscoUQ (Jiang et al., 2026) -- *Disagreement for Uncertainty Quantification**

| Aspect | DiscoUQ | DGAD |
|---|---|---|
| **Disagreement Role** | Quantifies uncertainty -> selective prediction (abstain) | **Routing signal** -> escalate to *different methodology* verifiers |
| **Action on Disagreement** | "I don't know" (abstain) | "Let me check with behavioural probe and LLM judge" |
| **Channels** | 5 role-specialized LLM agents (same architecture) | 5 **theoretically distinct** channels (perplexity, embedding, offset, perturbation, judge) |
| **Calibration** | Logistic regression on structure features | **Per-channel Platt/Isotonic calibration BEFORE gate** (raw scores incomparable) |
| **Cost Model** | Runs 5 LLM agents *every query* | Cheap channels *always*, expensive *only on dispute* (~10-20%) |
| **Adversarial Loop** | None | **Self-adversarial loop hunts false agreements continuously** |
| **Scope** | QA/Reasoning benchmarks (MMLU, TruthfulQA) | **Security: jailbreak, prompt injection, indirect injection** |

**Why DGAD wins for security:** Security demands a *decision* (block/allow), not abstention. False agreement (all cheap detectors fooled same way) is the critical failure mode -- DGAD actively hunts it; DiscoUQ doesn't address it.

---

### *4.2 vs. PromptForest (Appleroll Research, 2026) -- *Ensemble Voting with Uncertainty**

| Aspect | PromptForest | DGAD |
|---|---|---|
| **Ensemble Type** | 3 lightweight classifiers (DeBERTa, ModernBERT, XGBoost) -- similar task | 5 **heterogeneous** channels: statistical, semantic, offset, behavioural, judge |
| **Aggregation** | Weighted soft voting -> single score | **No voting.** Agreement = decide; Disagreement = escalate to *different* verifiers |
```

Uncertainty	Std dev of predictions -> flag for review	Disagreement gate *is* the uncertainty mechanism; routes to stronger evidence
Calibration	Implicit via ensemble diversity	**Explicit per-channel calibration** (Platt/Isotonic) before any combination
Adversarial Robustness	Not evaluated	**Built-in adversarial calibration loop** (generates attacks on own detector)
Latency	141ms mean (all 3 models always)	~50ms cheap channels; +200ms only on escalation
Indirect Injection	Not addressed	**Dedicated canary tripwire**

Why DGAD wins: Voting ensembles assume similar failure modes. DGAD's channels fail *differently* (statistical vs semantic vs offset) -- disagreement reveals *which* attack slipped through.

---

*4.3 vs. CASCADE (Abasikeles-Turgut & Gumus, 2026) -- *Cascaded Hybrid Defense**

Aspect	CASCADE	DGAD
Architecture	Sequential: L1 (regex) -> L2 (embedding) -> L3 (LLM fallback)	**Parallel cheap channels** -> calibration -> **disagreement gate** -> escalation
LLM Role	Fallback for L2 uncertainties	**Escalation verifier** (Channel D) + **Behavioural probe** (Channel C)
Decision Space	ALLOW / REVIEW / BLOCK	PASS / ESCALATE / BLOCK (with rationale from Channel C/D)
Calibration	None (hard thresholds)	**Mandatory calibration layer** before gate
Channel E (Offset)	Absent	**Representation offset** -- detects distribution shift without labels
Self-Adversarial	None	**Continuous loop** finds false agreements
Indirect Injection	Not addressed	**Canary tripwire** (from Yi et al. 2023)
Evaluation	5,000 MCP samples, 31 attack types	**Synthetic + real benchmarks; adversarial regression suite in CI**

Why DGAD wins: CASCADE's sequential design means *every* non-blocked prompt hits the embedding model. DGAD's parallel design + calibration means most prompts decide at cheap channels; only disputes pay LLM cost.

---

*4.4 vs. JailGuard (Zhang et al., 2023/2025) -- *Mutation-Based Discrepancy Detection**

Aspect	JailGuard	DGAD
Core Idea	Attacks are less robust -> mutate input -> measure KL divergence of outputs	**JailGuard = Channel C only** (behavioural probe)
Scope	Universal detector (jailbreak + hijack, text + image)	**Full pipeline**: cheap channels -> gate -> Channel C *only on dispute*
Query Budget	4-8 variants *per request*	**0 queries for 80-90%**; behavioural probe only on contested prompts
Access Required	Query access to target LLM	**Fully black-box** for Channels A/B/E (no target LLM queries)
Calibration	Fixed threshold (0.02) per model	**Per-channel calibration + gate thresholds**
False Agreement	Not addressed	**Adversarial loop explicitly hunts false agreements**
Multi-Modal	Yes (text + image)	**Text-first**; image/multimodal out of scope (per synopsis)

Why DGAD wins: JailGuard's mutation is expensive and requires target access. DGAD uses the *same perturbation-consistency principle* but **gates it behind cheap channels** -- only ~15% of prompts trigger Channel C.

---

*4.5 vs. ZEDD (Sekar et al., 2026) -- *Zero-Shot Embedding Drift Detection**

Aspect	ZEDD	DGAD

Signal	Embedding drift (cosine similarity) between clean/suspect pairs	**Channel E (offset) + Channel B (semantic)** -- similar but broader
Method	Fine-tune encoder -> GMM/KDE on drift scores	**Frozen encoders** (no fine-tuning); calibration + gate
Ensemble	Multiple embedding models -> flagging	**5 heterogeneous channels**, not just embedding variants
Calibration	GMM/KDE thresholding	**Platt/Isotonic per channel** -> disagreement gate
Statistical Channel	None	**Channel A: Windowed perplexity** (catches GCG-style suffixes ZEDD misses)
Behavioural Probe	None	**Channel C: Perturbation-consistency**
LLM Judge	None	**Channel D: LLM-as-judge with rationale**
Adversarial Loop	None	**Continuous self-red-teaming**
Indirect Injection	Email-centric (LLMail-Inject)	**Canary tripwire** generalizes beyond email

Why DGAD wins: ZEDD is a *single-signal* detector (embedding drift). DGAD fuses 5 signals with calibration; embedding drift is just one channel. DGAD catches GCG suffixes (perplexity) that ZEDD would miss.

---

*4.6 vs. SmoothLLM (Robey et al., 2023/2025) -- *Randomized Smoothing Defense**

Aspect	SmoothLLM	DGAD
Type	**Defense** (mitigates jailbreaks at inference)	**Detector + Router** (decides before target LLM)
Mechanism	Perturb input N times -> majority vote on target LLM	**Channel C perturbs -> checks consistency** (no target LLM for cheap channels)
Cost	N queries to target LLM *every request*	**Cheap channels: 0 target queries**; Channel C: few queries *only on dispute*
Theoretical Guarantee	Yes (under k-unstable assumption)	**Empirical + adversarial loop validation**
Fluent Attacks	Weak (character-level only)	**Channel B (semantic) catches AutoDAN-style fluent attacks**
Calibration	None	**Full calibration pipeline**
Indirect Injection	No	**Canary tripwire**

Why DGAD wins: SmoothLLM is a *defense wrapper* around the target LLM. DGAD is a *pre-filter* that decides *before* the target LLM sees the prompt. Different threat model placement.

---

*4.7 vs. FJD (Wang et al., 2025) -- *Free Jailbreak Detection**

Aspect	FJD	DGAD
Access	Requires logits (white-ish box)	**Fully black-box** for Channels A/B/E
Signal	First-token confidence shift with affirmative prefix + temperature	**5 signals**: perplexity, embedding, offset, behavioural, judge
Cost	~1 forward pass (but needs logit access)	**~50ms for 3 cheap channels** (no logits needed)
Calibration	Temperature scaling (heuristic)	**Platt/Isotonic per channel**
Scope	Jailbreak only	**Jailbreak + Prompt Injection + Indirect Injection**
Adversarial	Not evaluated	**Adversarial regression suite + self-adversarial loop**

Why DGAD wins: FJD needs logit access (not available on most APIs). DGAD works with API-only models. FJD is single-signal; DGAD's multi-channel design handles attack diversity.

4.8 vs. Llama Guard / Guardrail Benchmarks (Inan et al. 2023, Zizzo et al. 2025)

Aspect	Llama Guard / Benchmarks	DGAD
Approach	Instruction-tuned classifier (7B params)	**Channel D = LLM judge** (one of 5 channels)
Cost	7B model *every request*	**LLM judge only on ~15% escalation**
Calibration	Not emphasized	**Calibrated scores from all channels**
Over-Defense	Significant (Li & Liu 2024)	**Tracked separately** (NotInject-style benign FPR)

Adversarial Robustness	Vulnerable (Hackett et al. 2025)	**Self-adversarial loop hardens own detector**
Explainability	Binary safe/unsafe	**Channel D provides rationale; gate shows which channels disagreed**

Why DGAD wins: Llama Guard is *expensive always-on*. DGAD uses LLM judge *sparingly* and only when cheaper signals conflict.

4.9 vs. Statistical Baselines (Alon & Kamfonas 2023, Jain et al. 2023)

Aspect	Perplexity Filtering / Baseline Harness	DGAD
Signal	Windowed perplexity (GPT-2)	**Channel A = this exact method**
Limitation	Blind to fluent attacks (AutoDAN)	**Channel B (semantic) covers fluent attacks**
Threshold	Global hard threshold	**Calibrated probability + disagreement gate**
Multi-Signal	No	**5 channels fused via gate**

DGAD subsumes this: Channel A *is* windowed perplexity. The innovation is what happens *after* -- calibration, disagreement gate, escalation.

---

*4.10 vs. GradSafe (Xie et al., 2024) -- *White-Box Gradient Analysis**

Aspect	GradSafe	DGAD
Access	White-box (weights + gradients)	**Fully black-box**
Applicability	Open-weight models only	**Any LLM (API or local)**
Signal	Safety-critical parameter gradients	**5 black-box signals**
Cost	Backward pass per request	**Cheap channels: forward embeddings only**

Why DGAD wins: GradSafe is inapplicable to proprietary APIs (OpenAI, Anthropic, etc.). DGAD's black-box design works everywhere.

---

*4.11 vs. Adaptive Evasion (Hackett et al., 2025) -- *Attacking the Detector**

Aspect	Hackett et al.	DGAD
Direction	External red team attacks deployed guardrails	**Internal self-adversarial loop attacks own cheap channels**
Goal	Measure evasion success	**Generate training data for false agreements**
Integration	One-off evaluation	**Continuous CI/CD loop** (adversarial regression suite in CI)
False Agreement	Discovered incidentally	**Explicitly hunted as primary failure mode**

DGAD internalizes the threat: Instead of waiting for Hackett et al. to break your detector, DGAD breaks its *own* cheap channels continuously and learns.

---

5. Summary: What DGAD Uniquely Combines

Innovation	Prior Art	DGAD's Advance
Calibration before combination	No paper calibrates heterogeneous detectors before fusion	**Mandatory Platt/Isotonic per channel** -- raw scores incomparable
Disagreement = escalation (not voting/abstention)	DiscoUQ (abstain), PromptForest (vote), CASCADE (fallback)	**Routes to methodologically distinct verifiers**
Self-adversarial calibration loop	Hackett et al. (external eval)	**Continuous internal red-teaming -> training data**
False agreement as primary threat	Not explicitly modeled	**Adversarial loop optimizes for false agreement**
Cost-aware tiered invocation	SmoothLLM (always N queries), CASCADE (sequential)	**Parallel cheap -> escalate only disputes**

Five distinct detection theories	All papers: 1-2 signals	**Statistical + Semantic + Offset + Behavioural + Judge**
Indirect injection canary	Yi et al. (benchmark only)	**Operational tripwire in pipeline**
Fully black-box, model-agnostic	GradSafe (white-box), FJD (logits), JailGuard (queries)	**No target LLM access needed for Channels A/B/E**
Explainable decisions	Most: binary	**Gate shows disagreement; Channel D gives rationale**

---

6. Evaluation Methodology (DGAD-Specific)

Component	DGAD Approach	Prior Art Gap
Dataset Splits	Grouped by attack family + source (no near-duplicate leakage)	Most use random splits -> leakage
Holdout Set	Reserved, untouched until final eval	Often tuned on test
Metrics	ROC-AUC, PR-AUC, FPR (incl. NotInject over-defense FPR), ECE, escalation rate, cost/1K, p50/p95/p99 latency	Usually just accuracy/F1
Adversarial Regression	CI fails if previously blocked attack passes	Rarely automated
Ablation	Full grid: each channel, calibrated/uncalibrated, gate on/off	Limited ablations
Calibration	Reliability diagrams, ECE per channel	Often omitted

---

7. Conclusion

DGAD is not "another ensemble detector." It is a **disagreement-gated routing architecture** with:

1. **Calibration as a first-class primitive** -- not an afterthought
2. **Disagreement as actionable signal** -- not uncertainty to average away
3. **Self-adversarial hardening** -- not one-off red-teaming
4. **Cost-aware by design** -- expensive verifiers only on genuine ambiguity
5. **Black-box, model-agnostic** -- works on API-only models
6. **Complete threat coverage** -- jailbreak + direct injection + indirect injection

Every prior paper contributes *one piece* (perplexity, embedding drift, mutation, voting, cascading, LLM judge). **DGAD integrates all five detection theories into a calibrated, gated, self-hardening pipeline** -- the first to do so.

Appendix: Full Paper List (from Literature Review)

1. Zou et al. (2023) -- GCG
2. Perez & Ribeiro (2022) -- Ignore Previous Prompt
3. Liu et al. (2023) -- AutoDAN (Hierarchical Genetic Algorithm)
4. Su (2024) -- CoT-GCG
5. Inan et al. (2023) -- Llama Guard
6. Liu et al. (2023) -- Prompt Injection Formal Framework
7. Yi et al. (2023) -- Indirect Injection Benchmark
8. Alon & Kamfonas (2023) -- Windowed Perplexity Filtering
9. Jain et al. (2023) -- Baseline Defense Harness
10. Zizzo et al. (2025) -- Guardrail Benchmarking Harness
11. Li & Liu (2024) -- InjecGuard (MOF)

12. Xie et al. (2024) -- GradSafe
13. Hackett et al. (2025) -- Adaptive Evasion
14. Jiang (2026) -- DiscoUQ
15. Abasikeles-Turgut & Gumus (2026) -- CASCADE
16. Zhang et al. (2023/25) -- JailGuard
17. Appleroll (2026) -- PromptForest
18. Sekar et al. (2026) -- ZEDD
19. Robey et al. (2023/25) -- SmoothLLM
20. Wang et al. (2025) -- FJD

All papers documented in `DGAD_Literature_Review.xlsx` with working links.